

1. Newton_raphson

```
program newton_raphson
  implicit none
  real :: x, x_new, fx, dfx, tol
  integer :: iter, max_iter

  tol = 1.0e-6
  max_iter = 100
  x = 1.0

  print *, 'Iter    x          f(x) '
  do iter = 1, max_iter
    fx = x**3 - x - 1
    dfx = 3*x**2 - 1
    x_new = x - fx/dfx
    print *, iter, x, fx
    if (abs(x_new - x) < tol) exit
    x = x_new
  end do

  print *, 'Root = ', x
end program newton_raphson
```

2. Regular_falsi

```
program regula_falsi
  implicit none
  real :: a, b, c, fa, fb, fc, tol
  integer :: iter, max_iter

  tol = 1.0e-6
  max_iter = 100
  a = 1.0
  b = 2.0

  print *, 'Iter    a          b          c          f(c) '
  do iter = 1, max_iter
    fa = a**3 - a - 1
```

```

    fb = b**3 - b - 1
    c = (a * fb - b * fa) / (fb - fa)
    fc = c**3 - c - 1
    print *, iter, a, b, c, fc
    if (abs(fc) < tol) exit
    if (fa*fc < 0.0) then
        b = c
    else
        a = c
    end if
end do

    print *, 'Root = ', c
end program regula_falsi

```

3. Simpsons's_1/3

```

program simpson_one_third
    implicit none
    real :: a, b, h, sum
    integer :: i, n

    a = 0.0
    b = 1.0
    n = 10
    if (mod(n,2) /= 0) then
        print *, 'n must be even!'
        stop
    end if
    h = (b - a) / n
    sum = f(a) + f(b)

    do i = 1, n-1
        if (mod(i,2) == 0) then
            sum = sum + 2*f(a + i*h)
        else
            sum = sum + 4*f(a + i*h)
        end if
    end do

```

```

end do

sum = sum * h / 3.0
print *, 'Integral = ', sum

contains
  real function f(x)
    real, intent(in) :: x
    f = 1.0 / (1.0 + x*x)
  end function f
end program simpson_one_third

```

4. Simpsons's_3/8

```

program simpson_three_eighth
  implicit none
  integer :: i, n
  real :: a, b, h, sum, x

  a = 0.0
  b = 1.0
  n = 6 ! Must be multiple of 3

  if (mod(n,3) /= 0) stop

  h = (b - a) / n
  sum = f(a) + f(b)

  do i = 1, n - 1
    x = a + i*h
    if (mod(i,3) == 0) then
      sum = sum + 2.0*f(x)
    else
      sum = sum + 3.0*f(x)
    end if
  end do

  sum = sum * 3.0 * h / 8.0

```

```

    print '(F7.5)', sum  ! Only print final result to 5 decimal places
contains
    real function f(x)
        real, intent(in) :: x
        f = 1.0 / (1.0 + x*x)
    end function f
end program simpson_three_eighth

```

5. Trapezoidal

```

program trapezoidal
    implicit none
    real :: a, b, h, sum, x
    integer :: i, n

    a = 0.0
    b = 1.0
    n = 10
    h = (b - a) / n
    sum = (f(a) + f(b)) / 2.0

    do i = 1, n - 1
        x = a + i*h
        sum = sum + f(x)
    end do

    sum = sum * h
    print *, 'Integral = ', sum
contains
    real function f(x)
        real, intent(in) :: x
        f = 1.0 / (1.0 + x*x)
    end function f
end program trapezoidal

```

```
    end function f
end program trapezoidal
```

6. Jacobi

```
program jacobi
    implicit none
    integer :: i, j, n, iter, max_iter
    real :: a(3,3), b(3), x(3), x_new(3), tol, error

    n = 3
    tol = 1.0e-6
    max_iter = 100
    a = reshape([10.0, -1.0, 2.0, -1.0, 11.0, -1.0, 2.0, -1.0, 10.0], [3,3])
    b = (/6.0, 25.0, -11.0/)
    x = (/0.0, 0.0, 0.0/)

    print *, 'Iter      x(1)      x(2)      x(3) '
    do iter = 1, max_iter
        do i = 1, n
            x_new(i) = b(i)
            do j = 1, n
                if (i /= j) x_new(i) = x_new(i) - a(i,j)*x(j)
            end do
            x_new(i) = x_new(i) / a(i,i)
        end do

        error = maxval(abs(x_new - x))
        x = x_new
        print *, iter, x(1), x(2), x(3)
        if (error < tol) exit
    end do
end program jacobi
```

7. Gauss-seidel

```
program gauss_seidel
    implicit none
```

```

integer :: i, j, n, iter, max_iter
real :: a(3,3), b(3), x(3), x_old(3), tol, error

n = 3
tol = 1.0e-6
max_iter = 100
a = reshape([10.0, -1.0, 2.0, -1.0, 11.0, -1.0, 2.0, -1.0, 10.0], [3,3])
b = (/6.0, 25.0, -11.0/)
x = (/0.0, 0.0, 0.0/)

print *, 'Iter      x(1)      x(2)      x(3) '
do iter = 1, max_iter
  x_old = x
  do i = 1, n
    x(i) = b(i)
    do j = 1, n
      if (i /= j) x(i) = x(i) - a(i,j)*x(j)
    end do
    x(i) = x(i) / a(i,i)
  end do

  error = maxval(abs(x - x_old))
  print *, iter, x(1), x(2), x(3)
  if (error < tol) exit
end do
end program gauss_seidel

```

8. SOR

```

program sor_method
  implicit none
  integer :: i, j, n, iter, max_iter
  real :: a(3,3), b(3), x(3), x_old(3), w, tol, error

  n = 3
  tol = 1.0e-6
  max_iter = 100

```

```

w = 1.25
a = reshape([10.0, -1.0, 2.0, -1.0, 11.0, -1.0, 2.0, -1.0, 10.0], [3,3])
b = (/6.0, 25.0, -11.0/)
x = (/0.0, 0.0, 0.0/)

print *, 'Iter      x(1)      x(2)      x(3) '
do iter = 1, max_iter
  x_old = x
  do i = 1, n
    x(i) = (1 - w) * x(i)
    do j = 1, n
      if (i /= j) x(i) = x(i) - w * a(i,j) * x(j)
    end do
    x(i) = x(i) + w * b(i)
    x(i) = x(i) / a(i,i)
  end do

  error = maxval(abs(x - x_old))
  print *, iter, x(1), x(2), x(3)
  if (error < tol) exit
end do
end program sor_method

```